

**LAPORAN PRAKTIKUM
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III**



**Acara 9
OOP MVC Lanjutan ; Object, Dependency, Getter Setter**

**NAMA :
ANGELLO ERIYA DEWANGGA (E41250776)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

```

<?php
// Bertugas mengelola koneksi ke database menggunakan PDO
(singleton)
class Database
{
    private static ?Database $instance = null;
    private PDO $pdo;
    private function __construct()
    {
        $config = require ROOT_PATH . '/config/database.php';
        $dsn =
"mysql:host={$config['host']};port={$config['port']};dbname={$config
['dbname']};charset={$config['charset']}";

        try {
            $this->pdo = new PDO($dsn, $config['username'],
$config['password'], [
                PDO::ATTR_ERRMODE =>
PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]);
        } catch (PDOException $e) {
            http_response_code(500);
            exit('Koneksi database gagal. Periksa
config/database.php dan pastikan MySQL menyala.');
```

1. Membuat Class Database yang bertugas mengelola koneksi ke database menggunakan PDO

```
<?php

// Operasi CRUD data mahasiswa. Dependency (Database) di-inject
// Lewat constructor.
class MahasiswaRepository
{
    private PDO $pdo;

    public function __construct(Database $database)
    {
        $this->pdo = $database->getConnection();
    }
}
```

2. Membuat class MahasiswaRepository yang menerima objek Database melalui constructor (constructor injection) untuk melakukan operasi CRUD data mahasiswa.

```
<?php

class MahasiswaController extends Controller
{
    private MahasiswaRepository $repo;
    private ProdiRepository $prodiRepo;

    // Dependency di-inject lewat constructor (controller tidak
    // membuat koneksi database sendiri)
    public function __construct(MahasiswaRepository $repo,
        ProdiRepository $prodiRepo)
    {
        $this->repo = $repo;
        $this->prodiRepo = $prodiRepo;
    }
}
```

3. Membuat class MahasiswaController yang menerima objek MahasiswaRepository melalui constructor, sehingga controller tidak lagi membuat koneksi database secara langsung.

```
<?php

class Mahasiswa
{
    private string $nim = '';
    // ----- Getter -----
    public function getNim(): string
    {
        return $this->nim;
    }
}
```

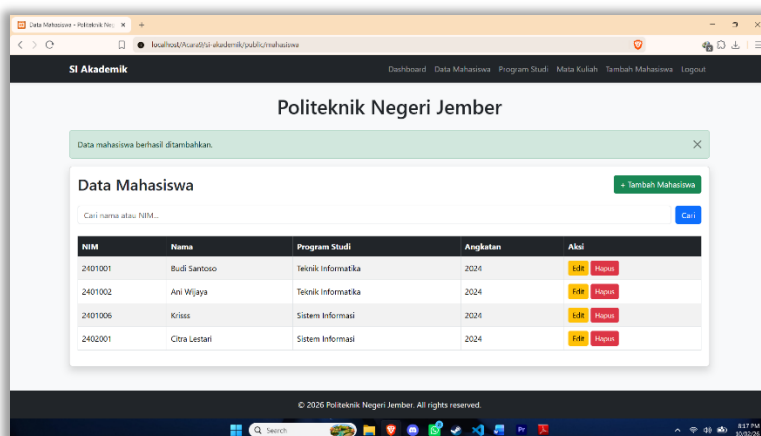
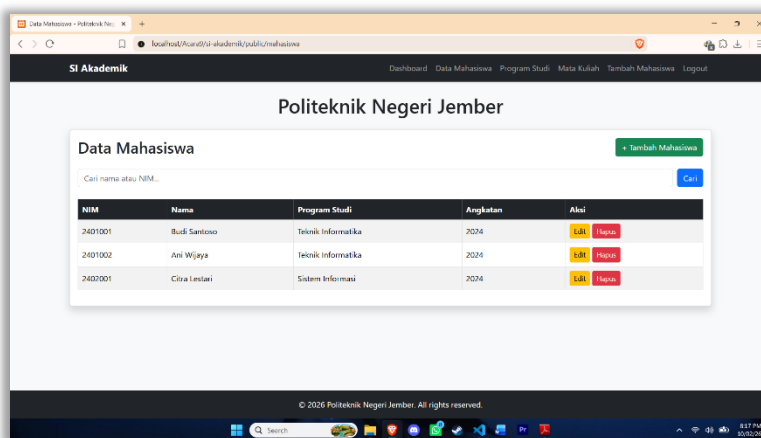
```

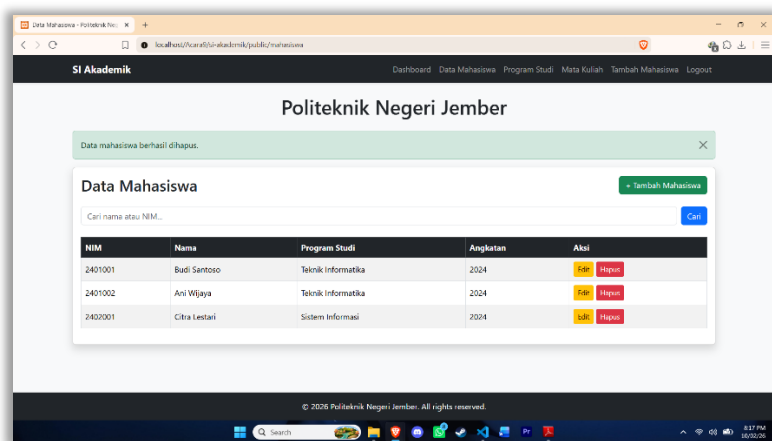
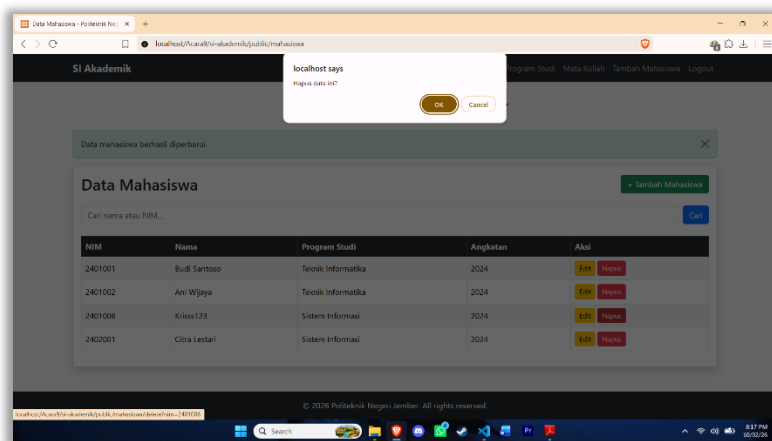
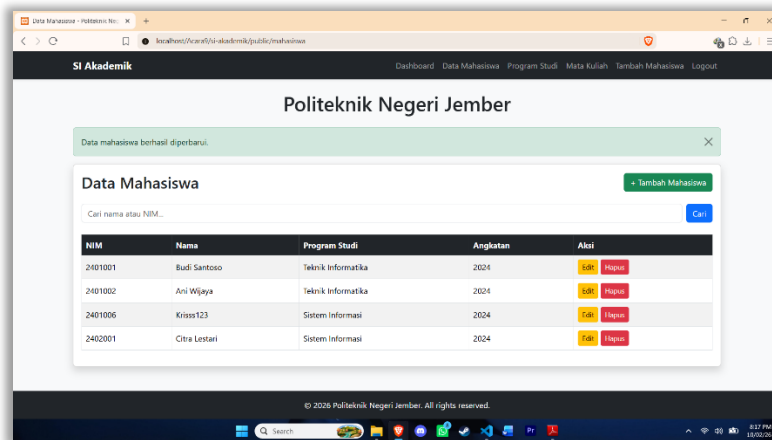
}
// ----- Setter + validasi -----
public function setNim(string $nim): self
{
    $nim = trim($nim);

    if ($nim === '' || !ctype_digit($nim) || strlen($nim) >
20) {
        throw new InvalidArgumentException('NIM wajib diisi
dan harus berupa angka (maksimal 20 digit).');
    }
    $this->nim = $nim;
    return $this;
}

```

4. Mengubah seluruh atribut pada class Mahasiswa menjadi private, kemudian membuat getter dan setter untuk mengakses serta mengubah data. Dan menambahkan Validasi NIM wajib di isi.





- Memastikan proses tampil, tambah, ubah, dan hapus data tetap berjalan dengan baik.